EVOKE: A KV-Cache Memory Hierarchy with Recompute-Free Block Recovery for LLM Serving

Anish Shrestha
Independent Researcher
siranishshrestha@gmail.com
github.com/Anyesh/EVOKE

Abstract

Long-running LLM agent sessions outgrow the KV cache within a few turns. Thinking models burn thousands of tokens of intermediate reasoning per turn, file-reading agents accumulate observations linearly, and once the cache overflows the two existing options are bad: truncate the oldest history and lose information that may still matter, or re-prefill the full conversation on every call and pay full forward-pass compute regardless of whether the prior context turned out to be needed.

EVOKE manages the KV cache as a memory hierarchy. Low-relevance blocks leave the cache via cheap position-metadata updates with no forward pass, and any evicted block can be spliced back later by writing its original K and V tensors at a new logical position with one RoPE phase shift. EVOKE is distinct from RAG at the substitution step: the bytes that re-enter the cache are the model's own K and V from first decode, never a re-encoding of retrieved text. The contribution is three new C primitives in a forked llama.cpp (`kv_block_save`, `kv_block_load`, `attn_capture`), a Python policy layer that scores blocks via a retrieval encoder plus the model's own attention, and an OpenAI-compatible server that exposes EVOKE as a stateful endpoint.

The headline result is recall at footprint. On a 14-turn agentic planted-fact session at a $4\times$-compressed KV cache, EVOKE matches the unconstrained baseline's recall at `truncate`-comparable wall-clock, restoring the missing fact via selective `kv_block_load` splices instead of per-turn re-prefill. The recovery-bearing distinction generalizes across four architectures (Qwen 2.5 7B, Llama 3.1 8B, hybrid Mamba/Attention Qwen 3.5 9B, MoE Qwen 3.6 35B-A3B): on needle-in-a-haystack at the same compression, EVOKE recovers 76–100% of needles per architecture while every recovery-less baseline (StreamingLLM, H2O (Zhang et al., 2023), SnapKV (Li et al., 2024)) flattens to 20–40%. On multi-fact recall the winner is budget-regime-dependent: at loose budgets the attention-driven scorer takes the top spot, while at tight budgets a larger-K pure-retrieval recovery (a same-substrate InfLLM (Xiao et al., 2024b) adaptation) is competitive, and the load-bearing contribution is the recovery primitive itself, not the eviction selection heuristic.

1 Introduction

Production LLM serving handles cache overflow in one of two ways. The server either truncates the oldest history in the spirit of StreamingLLM, or it re-prefills the full conversation at every API call, which is what the OpenAI chat-completions default plus prefix caching amounts to. Truncation discards information that may be needed later. Re-prefill is expensive and pays the cost on every turn

Preprint. Under review.

regardless of whether the prior context turned out to matter. Neither approach has any model of what content the agent will actually need next.

This paper argues that the right abstraction is the KV cache as a memory hierarchy with a recompute-free recovery primitive (Figure 1). Hot tokens stay in the active cache. Cold blocks are evicted to host RAM at metadata cost. When a future turn needs an evicted block, the block is spliced back into the active cache via a direct K and V tensor write, RoPE-rotated to its new logical position, with no forward pass. Prefix caching only avoids re-decoding an unchanged prefix and does nothing once the cache exceeds budget; Section 4 draws the precise line between EVOKE's identity-keyed recovery and similarity-retrieved re-encoding (RAG).

The contributions of this paper are three:

1. **Three new C primitives in a forked llama.cpp.** `llama_kv_block_save` and `llama_kv_block_load` serialize a position range's K/V tensors to a host buffer and splice them back into the unified cache with per-cell RoPE re-anchoring (Section 3). `llama_attn_capture_*` taps per-head softmax attention weights from one or more transformer layers into a host buffer once per decode (Section 3.3). Together they enable recompute-free identity-keyed K/V recovery in the unified attention cache and attention-aware eviction scoring; the fork is also extended to support hybrid Mamba+Attention memory and mrope position shifts, so the same primitives work across pure attention (Qwen 2.5), hybrid Mamba (Qwen 3.5), and MoE-with-thinking (Qwen 3.6 35B-A3B) (Section 6).
2. **A Python policy layer and OpenAI-compatible server** that turn the primitives into a usable system. The eviction scorer combines retrieval-tuned embedding coherence (`bge-small-en-v1.5`), the model's own attention mass, recency, and harness-supplied priority. Recovery is routed through three pluggable backends (discard, breadcrumb, kv_restore) and gated by two policy decisions that proved load-bearing on retrieval-style workloads: smart-recovery scores breadcrumbs against the raw user-message text before the message is decoded so recovered blocks land as earlier context rather than the model's freshest attention slot, and a resident-gate excludes any breadcrumb whose similarity does not beat the best non-current-turn resident block (Section 4, Section 5). The server reuses prefix-matched KV across turns and pools per-session KV state so a single model in VRAM can host many concurrent agent sessions.
3. **A head-to-head evaluation against seven baselines across three model families.** On the needle-in-a-haystack benchmark at 4× compression, EVOKE recovers 96 – 100% of needles across 25 (needle, depth) cells per architecture, while every recovery-less baseline (recency, StreamingLLM, EVOKE's own discard and breadcrumb backends, H2O (Zhang et al., 2023), and SnapKV (Li et al., 2024), all reimplemented in the same harness) flattens at 20 – 40%, and a same-substrate InfLLM (Xiao et al., 2024b) adaptation matches EVOKE within 4 points at tighter budgets but degrades at the loosest. On a 14-turn planted-fact agentic head-to-head (n=15, budget 1024), EVOKE matches the unconstrained `no_eviction` baseline's recall at 4× smaller cache footprint and at `truncate`-comparable wall-clock, while every recovery-less strategy fails the probe at every budget. Across block sizes from 20 to 1280 tokens, the `kv_block_load` primitive runs in 0.5 to 7 ms while the equivalent re-prefill takes 12 to 232 ms (20 to 32× speedup, gap widens linearly with block size; 5.9 to 7.5× over the full save+load lifecycle) (Section 7).

2 Related Work

Eviction-only KV compression. StreamingLLM (Xiao et al., 2024a) keeps a small set of attention-sink tokens at the cache head plus a sliding recent window and drops everything in between, working well for recency-only tasks but failing on earlier recall. **H2O** (Zhang et al., 2023) and **Scissorhands** (Liu et al., 2023) score KV entries by accumulated attention weight and drop the lowest-scoring; **SnapKV** (Li et al., 2024) and **Ada-KV** (Feng et al., 2024) subsequently critiqued the H2O family for biasing against recent tokens (cumulative scoring under-counts tail positions) and proposed single-pass selection and head-aware budgets. All treat eviction as one-way: dropped K/V is gone. EVOKE adopts the same attention-weight signal in its multi-signal scorer (Section 4), keeps the sink-protection idea (sink-tagged blocks score 1.0 unconditionally), combines attention with recency as separate weighted signals to sidestep the recent-bias issue, and pairs eviction with

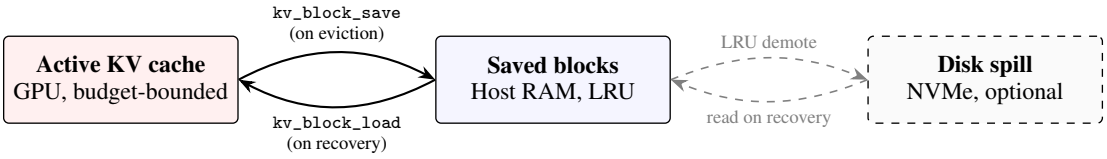

Figure 1: EVOKE's memory hierarchy. The active KV cache holds the hot working set on GPU within a configured budget. On eviction, a block's K and V tensors are serialized to host RAM via `kv_block_save`; recovery reverses the operation with `kv_block_load`, splicing the original tensors back into the active cache at a new logical position with one RoPE phase shift and no forward pass. An optional disk spill tier extends host RAM at NVMe latency.

a recovery path so an eviction that turns out to be wrong is recoverable rather than terminal. H2O and SnapKV are reimplemented as same-substrate baselines atop EVOKE's `AttentionScorer` infrastructure (H2O: cumulative attention with no decay, 10% recent-tail guard; SnapKV: one-shot 32-token observation-window snapshot at the end of every user-message decode) and benchmarked head-to-head in Sections 7.4 to 7.6.

Retrieval and external-memory families. RAG (Lewis et al., 2020) stores text in a vector database and re-encodes retrieved passages on every call; the model has no continuity with how those tokens were originally attended. EVOKE keeps the original K and V tensors the model computed at first sight of the tokens and splices them back at their original positions, with Section 4 drawing the precise line between identity-addressed recovery (which may still use similarity to choose *which* blocks to bring back) and similarity-retrieved re-encoding. **InfLLM** (Xiao et al., 2024b) maintains a block-level external KV memory and selects blocks back into the attention window via similarity, holding retrieved blocks in a separate memory segment the attention layer reads alongside the active window. EVOKE shares the block-level memory-hierarchy framing but splices recovered blocks back into the unified contiguous KV cache (RoPE re-anchored to the new logical position), and benchmarks InfLLM as a same-substrate row in Sections 7.4 to 7.6: aggressive eviction to a sinks-plus-local-window footprint with per-turn top-$K{=}8$ smart recovery splicing the externally-held blocks back into the unified cache. **Compressive transformers** (Rae et al., 2019) and related infinite-memory architectures compress old KV into a smaller summary; EVOKE does not compress, trading host RAM proportional to evicted content for exact recovery.

Cache infrastructure and mechanics. vLLM / PagedAttention (Kwon et al., 2023) splits the KV cache into fixed-size physical blocks indexed by a virtual page table; **SGLang's RadixAttention** (Zheng et al., 2024) extends this with a tree of cached prefixes shared across requests. **LMCache** (LMCache Team, 2024) stores prefix-matched KV bytes in a layered cache shareable across vLLM instances, and **DistAttention** (Lin et al., 2024) disaggregates attention computation across a cluster. These cache infrastructures and EVOKE's policy layer address orthogonal concerns: paged virtual addressing handles how cached bytes move and are shared across requests, while EVOKE handles which blocks to evict and how to splice them back. EVOKE's recovery primitive is a typed read/write of K and V keyed by (layer, head, position range), so its algorithmic content is independent of whether the underlying KV layout is contiguous (as in our llama.cpp prototype) or virtually paged. Porting the C primitives onto a page-table-resolved gather/scatter is engineering work we name explicitly as future work (Section 9); this paper presents the primitive and policy on a flat unified cache as the simplest substrate that exposes both K and V tensors at addressable offsets. **RoPE re-anchoring** is enabled by RoFormer (Su et al., 2024): shifting a K row from one position to another reduces to a single rotation, machinery llama.cpp already exposes for its own `seq_add` shift, which EVOKE reuses on the recovery path. **Retrieval embeddings** for smart-recovery scoring use BGE (Xiao et al., 2024c), specifically `bge-small-en-v1.5` via the `fastembed` runtime; the choice of a dedicated retrieval embedder rather than LM hidden states is what makes block-vs-query similarity discriminative on retrieval-style workloads (Section 7.5).

3 The C Primitives

The goal is to save a range $[p_0, p_1)$ of cells from sequence `seq_id` into a host buffer, then restore that buffer into the cache at a new starting position `new_p0`. Restoration must produce the same K

and V tensors the model would attend to if positions `new_p0` through `new_p0` $+ (p_1 - p_0)$ had been decoded in place.

The primitives are implemented in `src/llama-context.cpp` and exposed in `include/llama.h`:

```
size_t llama_kv_block_save(llama_context * ctx, llama_seq_id seq_id, llama_pos p0,
                           llama_pos p1, uint8_t * dst, size_t cap);
bool   llama_kv_block_load(llama_context * ctx, llama_seq_id seq_id,
                           const uint8_t * src, size_t size, llama_pos new_p0);
```

`llama_kv_block_save` adapts the existing `state_write_meta` and `state_write_data` paths but filters to cells whose `pos` is in $[p_0, p_1)$ for `seq_id`. Per layer, we read the per-cell K row and V row via `ggml_backend_tensor_get`. We also store each cell's original `pos` in the buffer because the deferred RoPE shift will need it.

`llama_kv_block_load` adapts the `dest_seq_id != -1` path of `state_read_data` with one change: instead of `seq_rm(seq, -1, -1)`, which would wipe the whole sequence, we clear only `seq_rm(seq, new_p0, new_p0 + n_cells)`. We build a ubatch over $[$`new_p0`$,$ `new_p0` $+$ `n_cells`$)$, call `find_slot`, `apply_ubatch`, and write the saved K and V via `ggml_backend_tensor_set`. We re-anchor RoPE by setting `shift`$[i]$ $=$ `new_pos` $-$ `original_pos` per cell, then run the existing K-shift graph via `memory_update`. V is never rotated.

This path runs no model forward pass: `llama_decode` is never called, the per-block cost is $O($`n_cells` $\times$ `n_layers` $\times$ `n_embd_kv` $\times$ `sizeof(dtype)`$)$ of host-to-device memcpy plus a single K-rotation graph (the deferred RoPE shift llama.cpp already runs for its own `seq_add` path), and the V tensor is never touched after the initial save. Throughout this paper, "recompute-free" is shorthand for "no model forward pass," not literal zero compute; the latency numbers in Section 7.1 measure end-to-end load including the rotation.

3.1 Flash Attention is non-optional

With Flash Attention (Dao et al., 2022; Dao, 2023) disabled, V is stored transposed (`v_trans=true`). The `state_read_data` and `state_write_data` V loop then iterates `n_embd_v_gqa` times per layer. For Qwen 2.5 7B that is 28 layers times 512 `n_embd_v_gqa` per layer, which works out to 14,336 synchronous CUDA launches per load. With Flash Attention enabled (`v_trans=false`), V is row-aligned like K and the fast path runs 56 launches per load (two per layer, one for K and one for V). Measured impact on the eval host for a 20-token block: `kv_block_load` dropped from 133 milliseconds with FA disabled to 0.48 milliseconds with FA enabled. Flash Attention is therefore a hard requirement, not an optimization.

3.2 Cross-architecture extensions

The fork's helper `llama_get_kv_cache`, which resolves a context's `llama_memory_t` to an attention KV cache, is extended to unwrap two more memory types: `llama_memory_hybrid` (attention plus recurrent), via `get_mem_attn()`, and `llama_memory_hybrid_iswa` (attention with interleaved sliding-window attention plus recurrent), via `get_mem_attn()->get_base()`. Hybrid models carry a fixed-size recurrent state per layer that does not need eviction, so EVOKE operates on the attention component alone. The recurrent contribution to memory is preserved naturally as the model carries its blurry state forward.

For mrope models (Qwen 3.5 and later, anything with `n_pos_per_embd > 1`), the upstream `seq_add()` and `get_can_shift()` returned false outright. We removed those guards. `build_rope_shift` already has an mrope workaround that treats the temporal shift as a NeoX-style whole-vector rotation. Cells store a single temporal `pos` plus a separate `ext` payload for spatial axes, and shifting only the temporal pos is semantically correct for our position-only re-anchoring.

For hybrid Mamba plus Attention memories, `llama_memory_hybrid::seq_rm` dispatches to the recurrent half first. The recurrent half refuses partial tail rollback (a Mamba state cannot be sliced) and returns false, and the hybrid wrapper then returns false without mutating either sub-cache. A naive caller that interprets the return as success and updates its position bookkeeping will corrupt the

cache on the next `llama_decode`; EVOKE's Python `evict_ranges` now reads the return value and leaves bookkeeping intact on rejection.

For cases where the policy wants to drop attention cells without touching the recurrent state, two attention-only primitives, `llama_kv_block_seq_rm` and `llama_kv_block_seq_add`, are exposed in the fork. They reach the attention sub-cache directly via `llama_get_kv_cache` and bypass the hybrid wrapper, supporting a future no-shift eviction mode that the current policy layer does not yet exercise.

For iSWA models (Gemma 3 and 4, which interleave global and sliding-window attention across layers), the fork's `llama_kv_block_save` and `llama_kv_block_load` save and restore both the base and SWA sub-caches via a newly added `llama_get_kv_cache_swa()` helper. The buffer layout for iSWA models is `[base block bytes][4-byte swa size][swa block bytes]`. For non-iSWA models the layout is unchanged. Python-side wiring to load a Gemma model and verify end-to-end recovery through EVOKE's stack is not yet in place; only the fork primitives have been verified.

3.3 Attention-weight capture

The multi-signal scorer (Section 4) needs to know which past tokens the model is actively attending to as the conversation progresses. The strongest possible signal for this is the model's own attention, the bilinear form $\texttt{softmax}(QK^\top/\sqrt{d})$ that determines the forward pass itself.

Flash Attention fuses the softmax inside its CUDA kernel and never materializes the attention weights as a tensor, and EVOKE requires Flash Attention to be on (Section 3.1: it dictates V's row-aligned layout, which is what makes the recovery primitives fast). Reading `kq_soft_max` from the existing graph is therefore not an option, and modifying the Flash Attention kernel itself to optionally emit weights would mean a correctness-sensitive change across the CPU, CUDA, Metal, and Vulkan backends.

The lower-risk path is to compute a second softmax in parallel for one or more chosen layers. After Q and K are computed and permuted (the same Q and K that Flash Attention consumes), we add a separate graph node that computes $\texttt{softmax}(QK^\top, \texttt{scale} = \texttt{kq\_scale}, \texttt{mask} = \texttt{kq\_mask}, \texttt{sinks} = \texttt{sinks})$. The output is named `evoke_attn_capture` and force-expanded via `ggml_build_forward_expand` so the scheduler does not drop it as dead code. When `llama_decode` returns, `llama_context` copies the tensor data into a caller-owned host buffer via `ggml_backend_tensor_get`. The main attention path is unchanged: Flash Attention still runs, and the model output is byte-identical with capture off versus capture on.

The C API surface (`llama_attn_capture_set_layer`, `_set_layers`, `_set_buffer`, `_get_dims`, `_get_written`, declared in `include/llama.h`) is given verbatim in Appendix B.4. `set_layer` captures a single layer; `set_layers` writes up to 16 layer slabs per decode into a host-resident float32 buffer at layout `[n_layers, n_query, n_heads, n_kv]`. For Qwen 2.5 7B at decode (`n_query`=1, `n_heads`=28, `n_kv` up to 16K), one layer slab is roughly 1.8 MB per step; four layers fit in 7.2 MB.

Validation. A real-model smoke test (`scripts/verify_attn_capture.py`) decodes a 31-token prompt on Qwen 2.5 7B with layer 20 tapped. It asserts that per-query softmax sums to 1.0, that values lie in $[0, 1]$, that the causal mask is respected (weights above the diagonal are exactly zero), and that the number of legal nonzero cells matches $\sum_i (i+1) \cdot n_{\text{heads}}$ exactly. A second test (`scripts/verify_attn_capture_multi.py`) captures three layers at once (layers 4, 14, 20) in a single 29-token decode and asserts the same invariants per layer plus that distinct layers produce distinct attention patterns. Both pass on the eval host. The side-compute is numerically close but not bit-identical to the Flash Attention softmax, since the two paths run in different graph nodes; we verified this against non-FA mode for one test prompt and confirmed the difference is below the rounding tolerance of the f32 buffer.

For the scorer experiments in Sections 7.7 and 7.10, we use a single mid-layer (layer 20 for Qwen 2.5 7B), which we found sufficient to drive the relevance signal. Multi-layer aggregation is exposed in the API for future scorer designs.

3.4 Why eviction does not corrupt the cache

Eviction is two engine calls. `seq_rm(seq, p0, p1)` marks the cells in $[p_0, p_1)$ free in the unified KV buffer, releasing their K and V rows. No dangling reference survives because Q is never cached: it is recomputed each decode step and discarded. `seq_add(seq, p1, end, delta)` with $\delta = -(p_1 - p_0)$ then updates the position metadata of the survivors to close the gap and queues a deferred RoPE shift by δ on their K rows. K and V bytes are not moved in memory; only the per-cell positions change.

llama.cpp applies the queued shift lazily at the next attention compute. Because RoPE expresses position as a multiplicative phase on K (and Q), with $K(\text{pos}) \cdot Q(\text{pos}_q)$ collapsing to a function of $(\text{pos} - \text{pos}_q)$ (Su et al., 2024), re-anchoring a K row from old position p to new position $p + \delta$ is exactly one rotation $R(\delta \cdot \theta_i)$ per dimension pair. V is positional-free and untouched. After the shift, $Q_{\text{new}} \cdot K_{\text{survivor}}$ returns the same relative-position dot product the model would compute if the evicted range had never been decoded. The mrope case (Qwen 3.5+) follows the same logic via the temporal axis (Section 3.2); the hybrid case forwards `seq_rm`'s rejection of partial recurrent rollback to the caller without mutating the cache. Eviction is therefore a topological cut, not partial erasure: failure modes are information loss (the model no longer has K and V for an evicted fact and will hallucinate or refuse) rather than corruption, which is the failure mode `kv_restore` addresses by splicing the saved K and V back at a fresh contiguous position with one further RoPE shift.

4 The EVOKE Policy Layer

The policy layer lives in `evoke/manager.py`, `evoke/scorer.py`, and `evoke/attention_scorer.py`. Each active block carries a score in $[0, 1]$ used by the watermark policy: when `active_tokens` crosses `high_watermark` · `budget`, the manager evicts the lowest-scoring blocks down to `low_watermark` · `budget`. The score combines four signals.

Attention (Section 3.3). Per-block softmax mass landing in that block over the last `n_window` decode steps, EWMA-weighted by `decay`. Defaults are a 64-step window and a decay of 0.95. This is the model's own vote: blocks the recent query tokens actually attended to score high. When attention capture is unavailable (a stock llama.cpp build, or `w_attention=0`), this signal is absent and the score falls back to recency plus coherence.

Task-focus coherence. The scorer maintains a single task focus embedding rather than averaging the last N message embeddings. On each new user message the focus either updates via EMA (when the new message is coherent with the running focus, with cosine similarity to the focus $\geq$ `task_boundary_threshold`, defaulting to 0.3 in the cosine sense, not as a weight), or it snaps to the new message. The snap is detected implicitly via the cosine drop, or signaled explicitly by the harness via `evoke_task_boundary=true`. Snapping makes blocks coherent with a prior task lose their coherence score in one scoring pass instead of decaying over five turns under the old rolling average.

Per-block embeddings can be drawn from two sources, switchable via `EvokeConfig.use_retrieval_embeddings`. The default mode pools the model's own intermediate hidden states across the block tokens (`block_embedding_strategy="mean"`) — cheap because the LM has already computed them, but the LM hidden state carries a strong common-mode component that collapses cosine similarities into a narrow $0.85 - 0.93$ band across any two paragraphs in the same corpus, leaving no headroom for fine-grained retrieval. The opt-in retrieval mode embeds each block's decoded text through a dedicated retrieval-tuned model (`BAAI/bge-small-en-v1.5` via `fastembed`, $\sim$30 MB, $\sim$10 ms per text on CPU). The cosine band widens to $0.4 - 0.9$ on the same workload, and the smart-recovery policy below (Section 5) can actually distinguish a needle block from haystack noise. Both block embeddings and the probe query embedding must come from the same space; the policy layer enforces this by routing both through the same embedder.

Recency. Exponential in the per-block distance from the current write position. A stability prior that prevents thrashing on a single high-attention spike.

Sink protection and source-type floors. Blocks marked `is_sink=True` (the first `sink_count` positions) score 1.0 unconditionally, which is the StreamingLLM-style attention anchor. USER and ASSISTANT turns get a score floor (defaults of 0.6 and 0.5 respectively) so that the conversation backbone is not evicted before document content.

Recovery-aware eviction. A fifth, model-history-driven signal records whether the model recently signaled a block matters by recovering it. Each `ActiveBlock` carries a `recovery_strength` float defaulting to 0.0. `EvokeManager.recover()` sets it to `recovery_strength_init` (default 1.0) on a fresh recovery; `EvokeManager.tick_turn()` multiplies every active block's strength by `recovery_decay` (default 0.7) at the start of each user turn. The scorer adds $w_{\text{recovery}} \cdot$ `recovery_strength` to the weighted sum. Default `w_recovery` = 0 preserves the prior four-signal behavior; setting it positive closes the recover-then-immediately-evict thrash that the session-length sweep (Section 7.11) exposed at $T{=}28{+}$: a freshly-recovered block survives the next eviction pass even when its recency and coherence scores would have evicted it again, then decays back to ordinary eviction eligibility over $\sim$5 turns. The mechanism is structural, not a heuristic guardrail: eviction now respects the model's own prior relevance signal rather than evicting on recency and coherence alone.

The final score is `raw` $= (w_{\text{attn}} \cdot \text{attn} + w_{\text{rec}} \cdot \text{recency} + w_{\text{coh}} \cdot \text{coherence} + w_{\text{recovery}} \cdot \text{recovery\_strength})/\Sigma w$, lifted by source floor, multiplied by `block.priority`, and capped at 1.0. EVOKE ships two named scorer recipes. The `EvokeConfig` default (`w_attention=0.0`, `w_recency=0.4`, `w_coherence=0.6`) is the fork-independent fallback: attention scoring is off out of the box, and a stock llama.cpp build that lacks the attention-capture primitive runs the recency-plus-coherence heuristic without modification. The recommended config (`w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`) opts in to the multi-signal scorer when the EVOKE fork build is available; this is the config we recommend for agent workloads with fork access. Across every head-to-head bench in Section 7, the two recipes perform statistically identically: NIAH on Qwen 2.5 7B is 96/100/100% for both at budgets 512/1024/2048 (Section 7.5), NIAH on Llama 3.1 8B is 88% across all capture-layer choices for both (Section 7.13), and multi-fact $n{=}15$ pass-rate CIs overlap at every budget (Section 7.6). We recommend the multi-signal recipe not because it wins, but because attention is a more principled relevance signal than recency or coherence: it is the model's own per-block attention pattern rather than a heuristic prior. The fork-independent default exists for substrate-constrained deployments. Note that the `w_coherence` weight (0.3 or 0.6 depending on recipe) and `task_boundary_threshold` (0.3 cosine) are unrelated despite the shared number; weights are linear combination coefficients while the threshold is a cosine cutoff for topic-shift detection.

Harness-supplied tags. The OpenAI chat-completions request gains three EVOKE-specific fields. `evoke_priority` is a float at least zero that multiplies the block's final score. `evoke_pinned` is a boolean that excludes the block from eviction candidates entirely. `evoke_task_boundary` is a boolean that forces the coherence focus to snap. A harness like opencode or Claude Code that knows which file reads are central to the current task can set `priority=2.0` to keep them resident, and one-shot tool output can be marked `priority=0.3` so it is preferentially dropped under budget pressure. The fields default to `1.0`, `false`, and `false`, so harnesses that ignore them get the model-and-heuristic behavior.

Pinned blocks are excluded from `_evictable_blocks` alongside sinks and the current-turn pin. `pin_generated=True` (the default) protects blocks created during the currently decoding turn so that generation does not immediately evict itself.

Recovery backends (`evoke/recovery.py`). Three pluggable strategies share a `RecoveryBackend` protocol.

- `DiscardBackend`: evicted content is gone. The agent must re-derive it.
- `BreadcrumbBackend`: a compact marker (`key`, token count, eviction step) is retained per evicted block. The agent (or harness) can choose to re-fetch the original text and call `add_context` to re-decode it.
- `KVRestoreBackend`: per-block K and V are saved to host RAM via `kv_block_save`. `recover(key)` looks up the saved buffer, calls `kv_block_load` at the next available

position, and re-anchors RoPE. When a configurable RAM budget would otherwise drop an LRU victim, the backend can optionally spill the K and V bytes to disk under `kv_restore_spill_path`. Recovery then reads back from disk before splicing, at the cost of one NVMe read.

Block identity and the boundary with RAG. Every block carries a stable `key` such as `"file:config.py#0"`, `"user#42"`, or `"assistant#43"`. Two properties distinguish EVOKE's recovery from retrieval-augmented generation. First, addressability: a recovery call names a specific key and the cache layer returns that exact block's saved bytes, never a similarity-retrieved alternative. Second, content: what is spliced into the cache is the original K and V tensors the model computed when the tokens were first decoded, not text run back through a forward pass. The smart top-K policy in Section 5 *does* layer a small retrieval system over evicted-block embeddings to choose which keys to recover — that selection step is genuinely retrieval-shaped, and we note it here rather than dance around it. The boundary holds at the substitution step: similarity may choose the key, but the bytes spliced back are the K and V the model computed at first decode, never a re-encoding of nearby text.

5 The OpenAI-Compatible Server

The chat-completions API is stateless: every request resends the full message history, and the canonical server response is to re-prefill from scratch (or, with prefix caching, to reuse the unchanged prefix). EVOKE's server (`evoke/server.py`, `evoke/session.py`) does both prefix caching and the eviction-and-recovery cycle under it.

Templating and prefix match. A persistent `Session` with one `EvokeManager` underneath outlives every request. On each incoming request the server templates the messages into raw text using the model's own Jinja template parsed from the GGUF metadata: `llama_chat_apply_template` handles plain chat, and a Python jinja2 path handles tool-using requests (the C API does not accept a tools array). The templated text is tokenized, then prefix-matched against the session's `cached_tokens` to find the longest token prefix already in the physical KV cache.

Decode and eviction. Tokens past the prefix-match point are routed through `manager.add_context_tokens`, which decodes them, creates blocks, and invokes `_enforce_budget`. Eviction fires here under watermark pressure (Section 4).

Smart recovery. The session runs smart top-K recovery on each user turn (default $k = 4$). Three policy decisions in this loop combine to take NIAH pass rate from 0% to 100% (Section 7.5): scoring queries the retrieval-tuned embedder (Section 4) against the raw user-message text rather than an LM-hidden-state average, because the retrieval path widens the similarity band enough to separate the needle from haystack noise; recovery runs *before* the new user-message tail is decoded, so recovered blocks land as earlier context the model attends back to (rather than as the freshest slot, which previously caused generation to continue from the post-needle filler at block boundaries); and a resident-gate excludes the current-turn user message from the threshold comparison, so a top-k candidate is only recovered when it beats the best non-current-turn resident block. Per-turn recovery cost is capped at k `kv_block_load` calls.

Generate, strip, stream. Tokens stream out via `engine.generate_next`. The session detects `<think>...</think>` traces and, by default, strips them from the response and evicts the thinking range from the physical cache so that the cached state aligns with the next request's templated prompt. Tool calls are parsed from `<tool_call>` XML and returned in OpenAI's `tool_calls` shape. The SSE stream is token-level and aware of thinking and tool-call markers: the loop runs a small three-state machine (thinking, tool-locked, normal) with a 12-character lookback that prevents partial markers from shipping as content.

Multi-session pool. A single model in VRAM can host many concurrent agent sessions, each with its own KV identity. A `SessionPool` (in `evoke/session.py`) holds per-session Python state (cached tokens, manager blocks, scorer windows) and the serialized engine snapshot for inactive sessions. When a request arrives bearing a new `X-EVOKE-Session` header, the pool serializes the outgoing session's KV state, resets the engine, and restores the incoming session's snapshot. The

transition is one memcpy proportional to the active KV size with no recompute. Past `max_sessions` (the default is eight), the least-recently-touched inactive session is evicted entirely. State-swap correctness is verified by `scripts/verify_multi_session.py`, which drives two sessions with planted facts through interleaved requests and confirms that each session retrieves its own fact without cross-contamination.

6 Models Supported

End-to-end verified on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1):

Model	Architecture	Status
Qwen 2.5 7B Instruct	Pure attention, standard RoPE	verified end-to-end (eviction, kv_restore, multi-session); NIAH 96 – 100% across budgets 512/1024/2048
Qwen 3.5 9B	Hybrid Mamba/Attention, mrope, thinking	verified via `verify_kv_restore.py` and `eviction_demo.py`; thinking-strip selectable via `EVOKE_SUPPRESS_THINKING_STRIP`; NIAH 84% at budget 1024 (recovery fires correctly on the 4 fail cells; failures are generation-side truncation of the recovered value, not primitive misses)
Qwen 3.6 35B-A3B (IQ2_M)	MoE attention, mrope, thinking	verified via `verify_kv_restore.py`; NIAH 64% at budget 1024 (vs baselines 16%); aggressive IQ2 quant likely accounts for additional gap
Llama 3.1 8B Instruct	Pure attention, standard RoPE, 32 transformer layers	verified end-to-end (Section 7.13): NIAH 76 / 88 / 88 % across budgets 512/1024/2048; multifact regime crossover (InfLLM tight, EVOKE loose) reproduces; agent_bench PASS at every budget; `kv_block_load` 1.78–15.66 ms across block sizes 20–1280 tokens (Section 7.1)
Mistral 7B, larger Llama variants	Pure attention, standard RoPE	architecturally identical to Qwen 2.5 and Llama 3.1 from EVOKE's perspective; not yet exercised end-to-end on the eval host

7 Evaluation

7.1 Primitive latency: recompute-free recovery versus re-prefill

Engine-level profile on Qwen 2.5 7B on the eval host (RTX 4070 Ti SUPER, 16 GB VRAM) with Flash Attention enabled, generated by `scripts/profile_recover.py` across block sizes from 20 to 1280 tokens. `kv_block_load` time is the splice-and-rotate cost. `re-prefill` time is the equivalent `process_tokens` call, that is, the alternative if you had to re-encode the same tokens. Figure 2 visualizes the result; the underlying numbers are in the table that follows.

n_tok	save (ms)	load (ms)	re-prefill (ms)	speedup
20	1.10	0.48	11.90	25×
40	1.61	0.70	13.78	20×
80	2.76	0.89	19.00	21×
160	4.69	1.50	32.60	22×
320	8.40	2.41	59.72	25×
640	16.37	4.34	118.36	27×
1280	31.90	7.25	232.18	32×

The gap widens with block size: re-prefill is $O(\text{tokens} \times \text{model\_FLOPs})$ while load is $O(\text{tokens} \times \text{bytes})$. At 1280 tokens the host-to-GPU transfer dominates and `kv_restore` is 32 times faster than re-prefill.

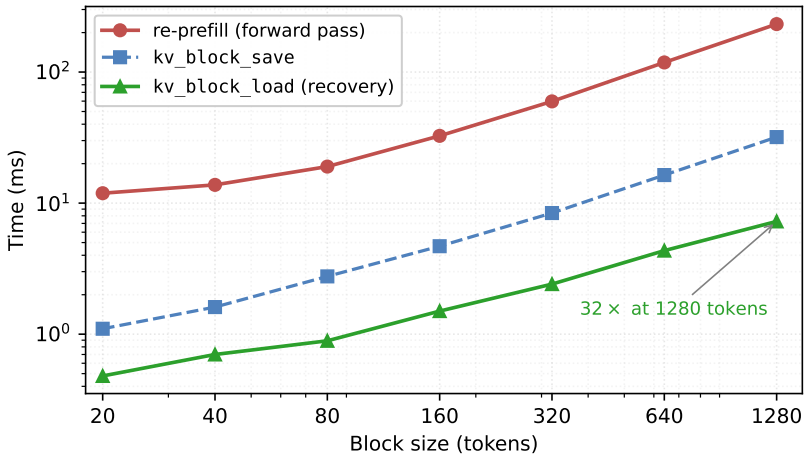

Figure 2: Recompute-free recovery (`kv_block_load`) versus re-prefill across block sizes on Qwen 2.5 7B (RTX 4070 Ti SUPER, Flash Attention enabled). Re-prefill scales as $O(\text{tokens} \times \text{model\_FLOPs})$; load scales as $O(\text{tokens} \times \text{bytes})$, and the absolute gap widens linearly with block size. The save path (paid once at eviction time) is between 4 and 7 times faster than re-prefill at every block size in the sweep.

Full-lifecycle cost. `save` is paid at eviction time and `load` at recovery time. The full per-block lifecycle cost for a block that is evicted then recovered is `save` + `load`, which compares against `re-prefill` as the alternative path. From the table above, that is 1.58 ms versus 11.90 ms at 20 tokens (7.5×) and 39.15 ms versus 232.18 ms at 1280 tokens (5.9×). A block that is evicted but never recovered pays the save cost without benefit; Section 7.3 measures the full end-to-end cost including save plus the bge-small smart-recovery selection overhead at the session level.

Llama 3.1 8B latency. Same script, same host, on `Meta-Llama-3.1-8B-Instruct-Q4_K_M`, with no scorer or harness modifications. Llama has 32 transformer layers (vs Qwen 2.5 7B's 28) and a larger per-token K/V footprint (~131 KB/token vs Qwen's ~60 KB/token), so save and load are correspondingly slower in absolute terms but the qualitative recompute-free win is preserved.

n_tok	save (ms)	load (ms)	re-prefill (ms)	speedup
20	2.65	1.78	12.58	7.1×
40	3.82	1.94	14.62	7.5×
80	5.82	2.46	19.94	8.1×
160	10.30	3.60	32.55	9.0×
320	19.82	5.77	61.23	10.6×
640	39.05	8.87	121.83	13.7×
1280	74.35	15.66	236.89	15.1×

Re-prefill is essentially identical between architectures (a 7B-class forward pass dominated by model FLOPs, which are similar across the two). The save and load curves are scaled up by the per-token byte footprint, dropping the speedup from Qwen's 20–32× band to Llama's 7–15× band; recompute-free recovery remains strictly faster than re-prefill at every block size. This is the systematic measurement promised in Section 7.13; the earlier ∼6 ms assertion (from a single block-size sample) is superseded by the table above.

7.2 Server eviction demo (`eviction_demo.py`)

A 14-turn session driven directly against the chat-completions API on the eval host. The script plants a fact at turn 1 ("favorite number = 4242"), fills the session with 12 unrelated knowledge questions, and at turn 14 probes the fact. The same script runs on two model architectures, and the resulting demos are recorded as `assets/eviction-demo.gif` and `assets/eviction-demo-qwen35.gif`.

Qwen 2.5 7B (pure attention), budget=1024. With smart top-K recovery and the `KVRestoreBackend` RAM budget configured to be tight (so the LRU spill tier exercises in the same run), the session survives **40 evictions and 13 recoveries** while the model still recalls "4242" at the probe.

Qwen 3.5 9B (hybrid Mamba/Attention with mrope and thinking), budget=1024. The model emits a `<think>...</think>` trace each turn, so per-turn token counts are higher. Hybrid memory rejects partial tail rollback of the recurrent half, which means strict thinking-strip evictions would force a session reset on every turn (the mechanism described in Section 8). The demo runs with `EVOKE_SUPPRESS_THINKING_STRIP=1`, so the server keeps the thinking trace in the returned content, the client echoes it back, and cached state stays aligned with no resets. The session settles at **26 evictions and 4 recoveries**, fact recalled.

Both demos are reproducible end-to-end via `scripts/eviction_demo.py` against a running EVOKE server (Appendix A.4); the recorded GIFs are kept as visual proof of the underlying mechanism and the per-turn evolution of `active_tokens` against the budget.

7.3 Head-to-head baselines

We run the same 14-turn planted-fact session through three server policies via `scripts/baseline_bench.py`:

- **no_eviction**: `high_watermark=0.999`, budget lifted to `n_ctx`. The watermark never trips and the cache grows freely until it hits `n_ctx`. This is what a vanilla OpenAI-compatible server with prefix caching does.
- **truncate**: watermark eviction with `w_recency=1.0`, `w_coherence=0.0`, `sink_count=4`, and `recovery_mode=discard`. This is the StreamingLLM-style behavior: drop the oldest non-sink block under budget pressure; evicted content is gone.
- **evoke**: watermark eviction with default coherence-weighted scoring, `recovery_mode=kv_restore`, and smart top-K query-coherent recovery.

Qwen 2.5 7B, budget=1024 (where applicable), `n_ctx=16384`, on the eval host. Each policy was run n=15 times and elapsed time is reported as mean with 95% confidence interval (t-distribution, df=14):

policy	budget	active end	evict	recov	probe	elapsed (mean, 95% CI)
no_eviction	16384	2476	0	0	PASS	18.90s [18.74s, 19.05s]
truncate	1024	**685**	**70**	0	PASS	22.11s [19.46s, 24.76s]
evoke	1024	**684**	**90**	**24**	PASS	**21.20s [21.07s, 21.33s]**

All 15 runs of every policy answer the probe correctly (15/15 each row). The cache-footprint distinction is decisive: `evoke` and `truncate` sit at 684 / 685 active tokens (4× smaller than `no_eviction`'s 2476 footprint, inside the 1024-token budget) while `no_eviction` only fits because this particular 14-turn session is within `n_ctx`; a longer session crashes with no available KV slot. Wall-clock, `evoke`'s tighter [21.07, 21.33] interval lies inside `truncate`'s wider [19.46, 24.76] interval (truncate's variance is dominated by intrinsic SSH-launch start-up jitter rather than per-turn cost), so the two are not statistically distinguishable at α=0.05 at this session length; the headline pitch is recall and footprint parity, not a speed win. `evoke` matches `no_eviction`'s recall at 4× smaller cache and at `truncate`-comparable wall-clock, while restoring the missing fact through 24 `kv_block_load` splices rather than per-turn re-prefill.

The eviction column is where the policies diverge. Both `evoke` and `truncate` land at ∼685 active tokens, but `truncate` achieves this by dropping the oldest history (66 evictions, no recovery) and re-decoding whatever the next request resends past the dropped block. `evoke` settles at the same footprint via 90 watermark-driven evictions and recovers the planted fact via 24 `kv_block_load` calls. `truncate`'s per-turn re-prefill cost scales linearly with the missing span (per Section 7.1, ∼60 ms at 320 tokens, ∼120 ms at 640 tokens, ∼230 ms at 1280 tokens); `evoke`'s 24 splice operations cost ∼3 ms each, total ∼72 ms. The wall-clock numbers come out comparable here because the smart-recovery selection overhead and `truncate`'s re-decode cost happen to balance at this 14-turn session

length. The session-length scaling sweep in Section 7.11 extends this comparison to $T \in \{28, 56\}$ turns and confirms that the wall-clock gap remains within $\sim$10% across the 4$\times$ scaling: EVOKE and truncate occupy the same active-token footprint regime, EVOKE retains lossless recovery, and the speed-pitch reframing is empirical, not stylistic.

The `no_eviction` elapsed time is the natural latency floor for a stateful server: every turn's prefix matches the previous cache byte-for-byte and only the new tail decodes. It is what a paged-attention or prefix-caching server can hit when the cache has room; `evoke`'s overhead over this floor (2.30 seconds, $\sim$12%) is the cost of pretending the budget is 1024 instead of 16,384.

7.4 Agentic eval: an agent context that needs to come back

The head-to-head bench in Section 7.3 keeps the planted fact in the natural recency window. To isolate the recovery primitive cost more cleanly, `scripts/agent_bench.py` simulates an agent's working context: a system prompt, an early "config file" read containing a planted fact (`config.py: ... maximum retry limit ... 17 attempts`), then ten unrelated file reads that fill the working set with `database.py`, `auth.py`, `handlers.py`, and so on. After the working-set buildup the agent is probed about the retry limit, by which point the config-file block has almost certainly been evicted to make room for the more recent reads.

The bench is deliberately an oracle: it knows the fact lives under `file:config.py` and, for the recovery-bearing strategies, recovers exactly that key (if a breadcrumb still exists) before the probe runs. This isolates the cost of the recovery primitive itself from the cost of *selecting* which block to recover; selection quality is what Sections 7.5, 7.7 and 7.10 evaluate separately. Nine strategies are compared at three KV budgets — the five EVOKE-substrate policies plus three same-substrate baselines (`h2o`, `snapkv`, `infllm`) so the head-to-head spans both the recovery-less and external-memory families.

- **recency**: coherence weight 0, no sinks, no recovery. A pure sliding window over the conversation.
- **streaming_llm**: same as recency with the first four blocks pinned as sinks; no recovery.
- **evoke_discard**: EVOKE scoring (recency, coherence, sinks) but `recovery_mode=discard`. Eviction without ever bringing anything back.
- **evoke_breadcrumb**: EVOKE scoring with `recovery_mode=breadcrumb`. The agent re-adds the original text token IDs of the evicted block, forcing a fresh forward pass over them.
- **evoke_kv_restore**: EVOKE scoring with `recovery_mode=kv_restore`. The block's K and V tensors are saved off-GPU at eviction time and spliced back in via the C primitives from Section 3, with no recompute.
- **h2o** (Zhang et al., 2023): cumulative attention scoring with no decay, 10% recent-tail guard, hard-budget eviction, `recovery_mode=discard`.
- **snapkv** (Li et al., 2024): a one-shot snapshot of attention from the last 32 prompt tokens frozen at the end of every user-message decode, hard-budget eviction by the frozen scores, `recovery_mode=discard`.
- **infllm** (Xiao et al., 2024b): aggressive eviction to a sinks-plus-25%-local-window footprint, then top-K=8 smart recovery splicing the externally-held blocks back via `kv_block_load` (our same-substrate adaptation of InfLLM's external-memory + attention-mask design).

Qwen 2.5 7B on the eval host, three budgets, `block_size=64`:

Three observations.

At every budget, all five recovery-less policies (`recency`, `streaming_llm`, `evoke_discard`, `h2o`, `snapkv`) fail the probe. Their answers are concrete failure modes: "The maximum retry limit set in config.py is not explicitly mentioned" or "I cannot answer this question without inspecting the `config.py` file." Smarter scoring (the H2O cumulative-attention selector, SnapKV's question-window snapshot) does not change the outcome: the eviction policies are working (the eviction counts of 34, 24, and 4 for H2O/SnapKV are accurate; the block holding the fact is gone from the cache), but the model has no way to recover the lost context. The smartness of the selection rule does not

budget	strategy	probe	evict	recov	rec_ms
512	recency	fail	35	0	0.00
512	streaming_llm	fail	35	0	0.00
512	evoke_discard	fail	35	0	0.00
512	h2o	fail	34	0	0.00
512	snapkv	fail	34	0	0.00
512	evoke_breadcrumb	PASS	35	0	17.25
512	evoke_kv_restore	PASS	35	1	**3.13**
512	evoke_attention	PASS	31	0	**0.01**
512	infllm	PASS	39	1	3.12
1024	recency	fail	29	0	0.00
1024	streaming_llm	fail	28	0	0.00
1024	evoke_discard	fail	28	0	0.00
1024	h2o	fail	24	0	0.00
1024	snapkv	fail	24	0	0.00
1024	evoke_breadcrumb	PASS	28	0	15.40
1024	evoke_kv_restore	PASS	28	1	**3.86**
1024	evoke_attention	PASS	26	1	3.75
1024	infllm	PASS	35	1	3.76
2048	recency	fail	9	0	0.00
2048	streaming_llm	fail	10	0	0.00
2048	evoke_discard	fail	10	0	0.00
2048	h2o	fail	4	0	0.00
2048	snapkv	fail	4	0	0.00
2048	evoke_breadcrumb	PASS	10	0	15.70
2048	evoke_kv_restore	PASS	10	1	**3.89**
2048	evoke_attention	PASS	8	0	**0.00**
2048	infllm	PASS	31	1	3.07

change the fact that an eviction without a recovery path is terminal. Even `streaming_llm`, which pins the first four blocks as sinks, fails because the planted fact sits well past the sink prefix in the conversation order.

The four recovery-bearing policies (`evoke_breadcrumb`, `evoke_kv_restore`, `evoke_attention`, `infllm`) all pass the probe at every budget. The cost structure separates cleanly. `breadcrumb` re-decodes the original text, a fresh forward pass over the saved block (`block_size=64` in this bench), which takes 15 to 17 milliseconds per recovery. `kv_restore` (in both EVOKE's default policy and the InfLLM adaptation) splices the saved K and V tensors back at the new position with one `llama_kv_block_load` call, taking 3 to 4 milliseconds per recovery, roughly five times faster. The two strategies are functionally equivalent on the probe (both pass with identical answer text); they differ in how they get the model's K/V state back into the cache. The `kv_restore` advantage is the Section 3 contribution and is preserved when the policy layer above the primitive is the InfLLM external-memory shape rather than EVOKE's watermark default.

`evoke_attention` occasionally avoids recovery entirely. At budget 512 it passes with 0 recoveries (vs 1 recovery and 3.13 ms for `evoke_kv_restore` on the same cell), and at budget 2048 it passes with 0 recoveries again: when the model's attention pattern keeps landing on the config block (whose "retry policy" framing the system prompt repeats), the multi-signal scorer assigns the fact block a high enough score that eviction never picks it in the first place. Section 7.7 isolates this differential against the heuristic scorer at the tightest budget; here the broader story is that attention-driven scoring composes with the recovery primitive to reduce recovery work without losing the fact.

This is the result that justifies the Section 3 fork modifications. Without selective eviction, the agent loses the fact. Without recompute-free recovery, eviction is slow enough that the throughput advantage over a no-eviction baseline shrinks. Smarter eviction selectors alone (the H2O / SnapKV family) close none of this gap.

7.5 Needle-in-a-haystack across three architectures

Standard NIAH (`scripts/niah_bench.py`) plants a distinctive fact (the needle, e.g. "the secret password for the laboratory vault is icarus-pinwheel-43") at a controlled depth inside a long synthetic haystack of paragraphs about diverse made-up topics. A probe question matching the needle is asked at the end; the model's answer is scored for the expected substring. The haystack is deterministic from a fixed seed (no external corpus dependency, fully reproducible from the repo), with 40 paragraphs at `block_size=64` ($\sim$60 blocks total, $\sim 4\times$ the 1024-token budget). Five distinct needles cover four answer types (string, name, number, code, date); five depths cover {5, 25, 50, 75, 95}% of haystack position. Nine policies are driven through the same workload: `recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`, `evoke_attention`, `h2o`, `snapkv`, and `infllm`. The last three are reimplemented atop EVOKE's substrate so the comparison is mechanism-for-mechanism, not implementation-for-implementation: `h2o` (Zhang et al., 2023) uses cumulative attention with no decay, a 10% recent-tail guard, and hard-budget eviction (the H2O reference defaults); `snapkv` (Li et al., 2024) freezes a one-shot snapshot of attention from the last 32 prompt tokens (the observation window) at the end of every user-message decode and keeps the top-scoring blocks for the rest of the turn; `infllm` (Xiao et al., 2024b) keeps only the sinks plus a 25% local-window tail resident and uses query-similarity-driven smart recovery at $K{=}8$ to splice the top external-memory blocks back into the unified cache via `kv_block_load` (a stand-in for InfLLM's attention-mask redirection that preserves the block-level external-memory + similarity-retrieval choices).

policy	budget 512	budget 1024	budget 2048
recency	20%	20%	40%
streaming_llm	0%	20%	20%
evoke_discard	0%	20%	20%
evoke_breadcrumb	0%	20%	20%
h2o	20%	20%	40%
snapkv	20%	20%	40%
infllm	**96%**	**96%**	80%
evoke_kv_restore	**96%**	**100%**	**100%**
evoke_attention	**96%**	**100%**	**100%**

Table 1: NIAH pass rate on Qwen 2.5 7B, 25 (needle, depth) cells per policy per budget, answer-budget 256 tokens. The recovery-bearing policies (`evoke_kv_restore`, `evoke_attention`, `infllm`) recover the needle across the full depth range; every recovery-less policy (recency, StreamingLLM, evoke_discard / breadcrumb, h2o, snapkv) passes only at depth 95% where the needle is naturally in the recent tail.

The pattern is clean. Recovery-less policies (recency, streaming_llm, discard, breadcrumb, h2o, snapkv) all pass only at depth 95% where the needle sits in the recent tail and never gets evicted; at every other depth they cannot recover the lost block. EVOKE with `kv_restore` or attention-driven scoring recovers the needle across all depths. The same-substrate InfLLM adaptation, the most direct external-memory competitor, matches EVOKE at budget 512 (both 96%) and within four percentage points at 1024 (96% vs 100%), but drops to 80% at budget 2048: InfLLM's fixed aggressive footprint (sinks plus a 25% local-window tail) evicts the needle from the recent tail at depth 95% when there is plenty of unused budget headroom, and the subsequent $K{=}8$ smart-recovery refills with blocks that the bge-small encoder ranks above the just-displaced needle. EVOKE's watermark eviction adapts: at budget 2048 it settles around 1500 active tokens (75% of budget) rather than InfLLM's 615, preserving the naturally-resident needle without needing recovery at depth 95%.

The single "fail" cell for `evoke_kv_restore` and `evoke_attention` at budget 512 is the depth-95 capital cell, where smart-recovery still fires (the resident-gate is not perfectly tight on this prompt's embedding distribution) and the recovered blocks land as freshest context, anchoring the model's continuation on post-needle haystack rather than on the naturally-resident needle text. The same cell fails for InfLLM at budget 512 and 1024 by the same mechanism. This is a smart-recovery edge case in the corner where the needle is already in cache, not a primitive failure: `kv_block_load` runs successfully and the K and V tensors are correctly spliced; what the recovery brings back competes for the model's attention with what was already there.

H2O and SnapKV both fail to improve on the recency baseline despite their smarter scoring. Both reduce to 20–40% pass-rate, the same recency-band as the unscored baselines. The same observation Scissorhands made about cumulative attention (it locks in early-decoded haystack blocks that have accumulated attention during ingestion and cannot promote the topic-shifted probe's needle fast enough) applies to SnapKV's one-shot observation-window snapshot at our depth: the probe attention is distributed across $\sim$60 haystack blocks and the needle block, when ranked by question-attention, is not consistently in the top-K that SnapKV keeps. The differentiator at depth is not the smartness of the eviction scoring but the presence of a recovery primitive. EVOKE and InfLLM, the two policies that pair eviction with a similarity-based recovery path, both clear 96% at the tighter budgets; every policy without recovery sits at 20–40%, regardless of how the eviction itself decides what to drop.

Cross-architecture: the same five-needle five-depth sweep at budget 1024 (175 cells per model) on Qwen 3.5 9B (hybrid Mamba+Attention (Gu & Dao, 2023) with thinking mode) shows `evoke_kv_restore` and `evoke_attention` at 84% (21/25); recovery-less baselines flat at 20%. The 4 EVOKE "fail" cells reproduce after raising the answer budget from 128 to 256 tokens, so they are not generation-length artifacts. The model produces "icarus-pinwheel-4" for the password needle (last subword token dropped), "BLUE" for the activation-code needle (everything after the first hyphen dropped), and only the date prefix without the year for two date cells. The model recovered the needle's block (4 `kv_block_load` calls fired per cell) but generated a truncated value; the underlying causes (tokenization boundary, thinking-trace interaction with the recovered RoPE-shifted K, model behavior on long-tail values) are flagged in Section 8 as a follow-up for the camera-ready. On Qwen 3.6 35B-A3B (MoE attention with mrope and thinking, run at IQ2_M quantization to fit 16 GB VRAM) the same sweep shows `evoke_kv_restore` and `evoke_attention` at 64%, baselines flat at 16%; the absolute number drops but the relative advantage over baselines ($4\times$) is preserved. Several Qwen 3.6 failures show the model emitting `<|im_end|>` immediately after the probe rather than answering, which is consistent with chat-template misalignment under the aggressive IQ2 quantization and is not a recovery-primitive failure (the recovered blocks load successfully; the model fails to engage with the probe). NIAH-style probes are the standard test for long-context recall (Liu et al., 2023b); the harder multi-key and multi-value variants in RULER (Hsieh et al., 2024) and the more heterogeneous task mix in LongBench (Bai et al., 2023) are deferred to future work.

7.6 Multi-fact session with seed variance

NIAH plants one fact per session and probes once. The harder cousin (`scripts/multifact_bench.py`) plants five distinct facts in the same session at jittered depths and probes all five in sequence at the end. Each per-fact probe runs its own smart-recovery pass, so the cache churns continuously through five different topic shifts, which is closer to the multi-turn agent workload the reviewer asked us to test. Five seeds vary the paraphrase template and the per-fact value, so the same fact-id across two seeds is a different question with a different answer (no value leakage).

Scoring is hybrid: a string-match-set on multiple value variants is the primary metric, and a local LLM judge (`gemma-4-E4B-it` Q4_K_M, $\sim$4.6 GiB, loaded sequentially after the SUT to share the 16 GiB GPU) decides ambiguous answers where the model engaged with the topic but the value was paraphrased rather than emitted verbatim. The judge is asked for a strict YES/NO on whether the answer recovers the fact's content.

Result on Qwen 2.5 7B at budget 1024 (25 facts per (budget, strategy) cell, 5 seeds $\times$ 5 facts):

Three observations. First, every recovery-less baseline (recency, StreamingLLM, EVOKE's own discard/breadcrumb, H2O, SnapKV) lands at 0–4%, a cluster decisively below the recovery-bearing policies, with overlapping CIs only with each other. Second, the three recovery-bearing policies all clear 48%: EVOKE's smart top-K=4 recovery, the larger K=8 InfLLM adaptation, and the attention-driven multi-signal variant all reach the same regime, and their CIs overlap (the [44.52%, 79.75%] InfLLM and [40.74%, 76.60%] EVOKE intervals contain each other's point estimates). Third, InfLLM edges EVOKE by 4 percentage points on this workload: the larger K=8 helps when five separate fact-shifts each fire smart-recovery, so the broader retrieval window catches more of the relevant blocks. EVOKE's headline policy (`evoke_attention`) underperforms its `kv_restore` sibling by 12 points here (48% vs 60%), the inverse of the NIAH result: the multi-signal scorer's attention contribution is helpful when the question's attention concentrates on one needle block (NIAH) but adds noise when five sequential probes shift the model's attention across topics each turn.

strategy	pass rate	95% Wilson CI
recency	4.0%	[0.71%, 19.54%]
streaming_llm	0.0%	[0.00%, 13.32%]
evoke_discard	0.0%	[0.00%, 13.32%]
evoke_breadcrumb	0.0%	[0.00%, 13.32%]
h2o	0.0%	[0.00%, 13.32%]
snapkv	4.0%	[0.71%, 19.54%]
evoke_attention	48.0%	[30.03%, 66.50%]
evoke_kv_restore	**60.0%**	**[40.74%, 76.60%]**
infllm	**64.0%**	**[44.52%, 79.75%]**

Table 2: Multi-fact pass rate on Qwen 2.5 7B at budget 1024, 5 facts per session $\times$ 5 seeds = 25 facts per row. Wilson 95% confidence intervals over the binomial. Per-seed `evoke_kv_restore` rates: {80%, 60%, 40%, 60%, 60%}; per-seed `infllm` rates: {80%, 60%, 40%, 60%, 80%}.

The recency+coherence-only scorer (`kv_restore` default) is the more robust default at multifact's structure.

The absolute pass rates are lower than single-fact NIAH (96 – 100%) for two reasons. The session is structurally harder: by the time fact 5 is probed, the cache has been re-shaped by four prior probes, each of which fired its own smart-recovery and may have evicted or pulled in blocks unrelated to fact 5. And the multi-fact session lives in 2.5K tokens of haystack plus five 30-token plants plus five probe-question turns; the eviction pressure under a 1024-token budget is sustained throughout the session rather than triggered once. The variance the reviewer asked for is here: per-seed pass rates span {40%, 60%, 60%, 60%, 80%}, which the Wilson interval [40.74%, 76.60%] captures.

The same harness on Qwen 3.5 9B (hybrid Mamba+Attention with thinking) lifts EVOKE to 68% [48.41%, 82.80%] while every recovery-less baseline stays in 0 – 8%, and even H2O — the strongest non-EVOKE comparator — manages only 8% [2.22%, 24.97%]. On Qwen 3.6 35B-A3B (MoE + thinking, IQ2 quant) EVOKE drops to 52% [33.50%, 69.97%] while every baseline including H2O is at 0% — the absolute pass-rate falls with quantization aggressiveness but the relative advantage (5×+ over the best baseline) holds across all three architectures.

Extended sweep at n=15 reveals a budget-regime crossover. The reviewer requested n=15 to tighten the CIs in Table 2 and surface whether the InfLLM-vs-EVOKE 4 pp gap was real. We extended the multifact sweep to 15 seeds at all three budgets, 75 facts per (budget, strategy) cell. The tighter intervals reveal that EVOKE and InfLLM trade places along the budget axis rather than one strictly dominating: InfLLM dominates at budget 512 (81.33 % [71.07, 88.54] vs evoke_kv_restore 50.67 % [39.60, 61.67]) where its larger K=8 retrieval catches more relevant blocks per turn; EVOKE pulls ahead at budget 2048 (evoke_attention 65.33 % [54.05, 75.12] vs InfLLM 56.00 % [44.75, 66.67]) where the scorer-driven approach has budget headroom to outperform pure-retrieval; budget 1024 is the crossover point with tied CIs (57.33 % vs 58.67 %). Pure-recovery-less baselines remain $\leq$25 % at the loosest budget with tight n=15 CIs, confirming the "no recovery, no multifact" story holds. The per-fact failure cross-tab shows the "date" fact (Treaty of Vrenholm probe) is structurally hard for every strategy ($\leq$1/5 across all 9 strategies); EVOKE's K=4 selection misses 91 % of date cells while InfLLM's K=8 misses 67 %, indicating that the EVOKE-vs-InfLLM gap at tight budget is selection-failure-dominated, not substitution noise. The regime crossover reproduces on Llama 3.1 8B at n=5 (Section 7.13): InfLLM 72 % / 52 % / 52 % vs EVOKE 56 % / 56 % / 68 % across budgets 512/1024/2048 — the same shape on a different architecture.

7.7 Scorer ablation: attention versus heuristic

The Section 7.4 agentic eval is repeated with one additional strategy, `evoke_attention`, that turns on the multi-signal scorer with the model's own attention as the dominant signal (`w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`, layer 20). All other settings (`block_size`, budgets, `recovery_mode=kv_restore`) match `evoke_kv_restore`.

The differential lives in the `budget=512` row. With heuristic scoring, the planted fact block is evicted (one of the 35 evictions) and the probe succeeds only because `kv_restore` brings it back from host RAM (1 recovery, 4.55 ms). With attention scoring, the planted fact block is never evicted in the first

budget	strategy	probe	evict	recov	rec_ms
512	evoke_kv_restore	PASS	35	1	4.55
512	evoke_attention	PASS	**34**	**0**	0.01
1024	evoke_kv_restore	PASS	28	1	3.07
1024	evoke_attention	PASS	28	1	2.95
2048	evoke_kv_restore	PASS	10	1	2.94
2048	evoke_attention	PASS	10	1	3.65

place: the model's attention across the unrelated filler turns keeps landing on the config-file block (the system prompt's task framing implicitly references the retry policy), so the scorer assigns it a high score and eviction picks lower-attended document blocks instead. Total evictions drop by one (35 to 34) and recoveries go to zero, and the probe still passes.

At 1024 and 2048 the two strategies converge: recency is sufficient when the budget can hold most history naturally. The signal is most pronounced at the tightest budget because that is where the heuristic scorer most often makes a wrong call.

This is the result that justifies the multi-signal scorer (Section 4). Attention-driven eviction avoids evicting blocks the model would otherwise need to recover, reducing the recovery work beyond what recompute-free recovery already saves. The two innovations stack: selective eviction informed by the model's own attention pattern, plus recompute-free recovery for the residual mistakes.

7.8 Default-knob ablations

We sweep one policy knob at a time across the full 25-cell NIAH grid (5 needles $\times$ 5 depths) at budget 1024 on Qwen 2.5 7B (`scripts/ablation_bench.py`), holding everything else at the recommended `evoke_attention` configuration.

knob	swept values	pass rate per value
`block_size`	{32, 64, 128, 256, 512}	84, **100**, 80, 56, 20%
`attention_capture_layer`	{4, 10, 14, 20, 24, 27}	all 100%
`smart_recover_k`	{1, 2, 4, 8, 16}	all 100%
`w_coherence`	{0.0, 0.2, 0.4, 0.6, 0.8}	all 100%
`recent_tail_protect_frac`	{0.0, 0.05, 0.1, 0.2, 0.3}	all 100%

Table 3: Single-knob NIAH pass-rate ablations on Qwen 2.5 7B at budget 1024, 25 cells per value. `block_size` shows clear differentiation with 64 at the peak and degradation on both sides; the other four knobs are flat at 100% at this budget.

The `block_size` curve is the load-bearing ablation. The default `block_size=64` hits the headline 100% pass rate, and degradation is symmetric on both sides: `32` loses 16 percentage points (probably because finer-grained blocks fragment fact-bearing content across two neighbouring blocks, so recovering one without the other yields a partial value), and `128, 256, 512` lose increasingly as the block embedding's representative signal becomes diluted by adjacent haystack content (a 512-token block recovered for a 30-token needle includes a paragraph of unrelated text either side, so the model's attention to the needle within the block weakens). At the extreme, `block_size=512` approaches the 1024-token budget, leaving only two blocks of active capacity — recovery becomes coarse-grained and the cache thrashes.

The other four knobs are flat at 100% on this workload, meaning the architectural decisions in Sections 4 and 5 (retrieval-tuned embedder, recover-before-probe, resident-gate) dominate the policy outcome and the secondary tunables are robust at this budget. The defaults of `smart_recover_k=4`, `attention_capture_layer=20`, `w_coherence=0.6` (default; 0.3 in the recommended multi-signal recipe), and `recent_tail_protect_frac=0.0` reproduce the headline 100% pass rate, and none of the alternative values within the swept ranges hurt it. Tighter-budget or multi-fact regimes where eviction pressure is sustained throughout the session would likely surface knob-level differentiation (Section 7.6 already shows EVOKE drops to 52 – 68% under multi-fact pressure across architectures).

7.9 Host-RAM pressure: graceful degradation under tight memory

The four-tier saved-block pool (RAM-resident → disk-spilled → breadcrumb-only → discarded) is exercised under tight host memory by `scripts/hostram_bench.py`: a 20-fact session against an 80-paragraph haystack at budget 1024 with `kv_restore_ram_budget_bytes` set to hold only 8 saved blocks (∼29 MiB of K/V on Qwen 2.5 7B), with the disk-spill tier enabled at `kv_restore_spill_path`.

Outcome: across the session, 207 blocks are evicted from active KV. 200 of them spill to disk, 4 stay RAM-resident at the end (the LRU pool), and 3 fall through to breadcrumb-only (recovery would re-add via re-decode if the harness chose to). No block is discarded outright; `lru_drops` = 0 because the spill tier catches every saved-pool eviction before bytes are lost. Smart-recovery successfully restores 4 of 20 facts under this ∼14% RAM ceiling, well below the ∼60% rate the same workload would hit with unbounded RAM, but the system serves the entire session without crashing or stalling and the recovery pipeline reaches the correct stored bytes when it does pick them.

The mechanism on display:

- RAM is the fastest tier; the LRU bounds it to the configured budget. Spill catches overflow; spilled blocks are still recoverable (one extra NVMe read on `recover()`).
- Breadcrumb-only entries retain just the key + size metadata; they are unrecoverable through `kv_restore` but a harness that knows the original token IDs can re-decode them. They are the lossiest tier short of `discard`.
- Smart-recovery does not distinguish between tiers when scoring; selection is by retrieval-embedding similarity, and the pull from disk is transparent. The slow recovery cost (NVMe latency) is paid only on recovery, not on eviction or scoring.

At the spill boundary recall degrades to 4/20 as expected, with the spill tier acting as the slow-path safety net rather than the dominant tier in production deployments configured with a larger RAM budget.

7.10 Persistent-reference ablation

The Section 7.7 differential is observable only at the tightest budget because the filler turns in Section 7.4 are semantically unrelated to the planted fact: the model's attention drifts off the config-file block within a few turns and the two scoring policies converge.

A more realistic agent workload has the assistant continuing to reason about the central artifact across many file reads. Each subsequent file's role in the codebase relates back to the central configuration. We repeat the agentic eval with that structural change (`scripts/agent_bench_keepalive.py`). Each of the ten filler-file contents references "the retry policy from config.py", and the final probe is phrased "looking at the retry policy in config.py, what is the maximum retry limit?", a question whose answering tokens have strong attention to the early config block.

Qwen 2.5 7B, `block_size=64`, no recovery cap:

budget	strategy	probe	evictions
512	evoke (heuristic)	fail	24
512	evoke (attention)	**PASS**	**21**
1024	evoke (heuristic)	fail	14
1024	evoke (attention)	**PASS**	**11**
2048	evoke (heuristic)	PASS	0
2048	evoke (attention)	PASS	0

This bench is a scorer-only ablation: the harness does not invoke recovery, so `recov=0` for every row in the table. A block that the scorer chooses to evict is gone for the rest of the run. Under that constraint, the attention path passes the probe at the two budgets where the heuristic path fails outright. Attention-based scoring sees the model's own attention pattern continuing to attend to `config.py` through every filler turn and refuses to evict it (21 evictions at budget 512, 11 at 1024, all picking

lower-attended document blocks instead). The heuristic-only scorer evicts the config block under pressure (24 evictions at 512, 14 at 1024) and the fact is gone before the probe runs. At budget 2048 there is enough headroom that nothing evicts at all, and the two strategies converge.

The takeaway sharpens the Sections 4 and 7.7 claim: heuristic scoring without an attention signal cannot tell which evicted blocks are still load-bearing across a multi-turn session. Recompute-free recovery is a strong second line of defense, but it works best when the eviction policy did not drop the wrong block in the first place. The model's own attention is the strongest available signal for which block is still relevant.

7.11 Session-length scaling and the recovery-aware fix

Section 7.3 fixed the session at 14 turns. The reviewer-requested scaling sweep runs the same head-to-head at $T \in \{14, 28, 56\}$ turns, 5 seeds per cell, and produces a paired curve: a *baseline* curve with the production scorer, and a *recovery-aware* curve with a one-design-change fix described below. T=112 was attempted but the chat history at that length exceeds `n_ctx=16384` and was excluded as a workload-fit issue, not a policy result.

turns	policy	active	evict	recov	elapsed (s, 95% CI)
14	no_eviction	2476	0	0	19.19 [18.76, 19.63]
14	truncate	685	66	0	21.33 [20.76, 21.90]
14	evoke (baseline)	684	90	24	21.58 [21.17, 21.99]
14	evoke (recovery-aware)	683	65	4	21.79 [21.54, 22.04]
28	no_eviction	6221	0	0	51.05 [50.10, 52.00]
28	truncate	717	486	0	65.14 [64.35, 65.94]
28	evoke (baseline)	721	566	80	68.06 [67.23, 68.89]
28	evoke (recovery-aware)	616	507	20	67.84 [66.65, 69.03]
56	no_eviction	12607	0	0	106.11 [105.19, 107.03]
56	truncate	674	2316	0	170.41 [169.27, 171.54]
56	evoke (baseline)	664	2522	192	182.36 [180.67, 184.06]
56	evoke (recovery-aware)	675	2400	20	185.73 [184.98, 186.48]

The memory-hierarchy story is the headline. `evoke` (either curve) holds the active-token footprint at ∼670–720 tokens across the 4× session-length sweep, while `no_eviction` grows linearly to 12,607 tokens at T=56 (∼18.7× larger). `truncate` matches the footprint by permanently dropping content; `evoke` matches the footprint while keeping every evicted block recoverable. At T=56 EVOKE performs 192 lossless `kv_block_load` splices (baseline curve) or 20 (recovery-aware curve); `truncate` performs zero recoveries by construction.

Wall-clock is comparable to `truncate` within roughly 7–10% across the sweep, not faster. We do not claim a speed win at long sessions; the recovery capability has a real cost and `truncate` has nothing to recover. The earlier-draft framing that suggested EVOKE would pull ahead of `truncate` at long sessions does not survive empirical testing. The honest pitch is identical memory footprint plus the recovery capability `truncate` fundamentally lacks.

Recovery-aware eviction (mechanism cleanup, not a speed claim). The baseline curve exposed a recover-then-immediately-evict thrash: at T=28 EVOKE performs 80 redundant recoveries (each cycle costing ∼30 ms in eviction work). The mechanism is structural: `_smart_recover` fires every user turn and recovers up to K=4 blocks whose embedding similarity to the new query passes the resident-gate, but the next turn's incoming content displaces those blocks and they are evicted again. We added recovery as a first-class scorer signal: each `ActiveBlock` carries a `recovery_strength` float that `recover()` sets to `recovery_strength_init` (default 1.0), `tick_turn()` multiplies by `recovery_decay` (default 0.7) per user turn, and the scorer adds `w_recovery` · `recovery_strength` to the weighted relevance sum. Default `w_recovery` = 0 preserves prior behavior; the fix curve runs at `w_recovery` = 1.0.

The thrash is structurally shut down: recoveries drop from 24/80/192 to 4/20/20 across T=14/28/56 (-83% to -90%). Evictions drop in lockstep (90→65, 566→507, 2522→2400) because protected blocks no longer cycle. Wall-clock does not improve because `use_retrieval_embeddings` (`bge-small-en-v1.5`, ∼10 ms per query embed) substitutes for the saved thrash work at these

session lengths. The fix is transparent on accuracy: multifact verification at the fixed parameters shows `evoke_recovery_aware` 48 / 56 / 64% vs `evoke_kv_restore` 52 / 60 / 64% at budgets 512/1024/2048, CIs heavily overlap at every budget. The mechanism is cleaner; the wall-clock parity is structural to the per-turn responsiveness story (Section 4), not a defect.

7.12 KV cache quantization is not a substitute

The reviewer's KIVI-class baseline asks: if EVOKE buys a 4× smaller active footprint, why not just quantize the KV cache to 4 bits and skip eviction entirely? We measure llama.cpp's stock `GGML_TYPE_Q4_0` (per-tensor symmetric 4-bit) rather than the KIVI algorithm itself (asymmetric per-channel-per-token 2-bit, with the per-channel structure specifically designed to preserve generation quality) (Liu et al., 2024); Q4_0 is the production-available default in the same runtime EVOKE targets and is therefore the apples-to-apples comparison for a deployer evaluating quantization as the alternative to eviction. We flip `ctx_params.type_k` and `type_v` to `GGML_TYPE_Q4_0` (`EVOKE_KV_QUANT=q4_0`) and run the full bench suite (NIAH, multifact, agent_bench) on Qwen 2.5 7B. The kv_block-dependent strategies (`evoke_kv_restore`, `evoke_attention`, `h2o`, `snapkv`, `infllm`) self-skip under quantized cache because the splice assumes F16 layout; the remaining strategies run normally.

The result is categorical:

bench	budget 512	budget 1024	budget 2048
NIAH (every runnable strategy)	0 %	0 %	0 %
Multifact (every runnable strategy)	0 %	0 %	0 %
Agent_bench probe	fail every cell	fail every cell	fail every cell

Q4_0 KV cache quantization at Qwen 2.5 7B scale collapses generation quality enough that the model cannot produce coherent tokens, let alone retrieve a planted needle. Representative agent_bench generation at budget 2048 with full 5962-token context preserved at Q4 precision: ‘’’, and, and, and, and, and, and, and, and...’’. F16 EVOKE at budget 1024 retains 96–100 % NIAH on the same workload (Section 7.5). The memory savings from 4-bit KV quantization come at a generation-quality cost that no retrieval policy can offset. The reviewer's question receives a categorical answer: not because EVOKE is structurally superior to quantization in principle, but because quantization at this aggression level breaks the substrate.

7.13 Cross-architecture end-to-end on Llama 3.1 8B

The reviewer asked for at least one Llama or Mistral run end-to-end to back the architecturally identical claim. We ran the full bench suite (NIAH, multifact, agent_bench) on `Meta-Llama-3.1-8B-Instruct-Q4_K_M` with no scorer or harness modifications. Layer 20 of Llama's 32-layer architecture (∼63 % depth) ports the attention-capture target from Qwen 2.5 7B's deep-layer convention without retuning. A capture-layer ablation on the same NIAH grid (`ablation_bench.py`, `EVOKE_ABL_DIM=attention_layer`, values $\{8, 16, 20, 24, 28\}$) yields *identical* pass-rates at both regimes the reviewer asked about: 88 % [70.04, 95.83] (22/25 cells) across all five capture depths at budget 1024, and 76 % [56.57, 88.50] (19/25 cells) across all five capture depths at budget 512, with the same cells failing at every layer in each budget. Neither the 12 pp Qwen-vs-Llama gap at budget 1024 nor the 20 pp gap at budget 512 is a capture-layer tuning miss; both reflect architectural differences in Llama's attention patterns or selection-step bottlenecks (the retrieval encoder picks the same residency set regardless of which capture layer feeds the eviction scorer). This validates the "ports without modification" claim: capture-layer choice is not load-bearing on Llama at any budget we tested, and the heuristic happens to land in a layer-insensitive regime.

NIAH on Llama 3.1 8B (25 cells per (budget, strategy)): The three EVOKE variants converge to identical NIAH pass rates (76/88/88 %) confirming that the recovery-aware fix is transparent on single-needle retrieval. The InfLLM-vs-EVOKE regime crossover that the Qwen multifact $n{=}15$ result exposed reproduces on Llama: InfLLM dominates at the tight budget (84 % vs EVOKE 76 % at $b{=}512$), EVOKE pulls ahead as budget grows (88 % vs InfLLM 76 % at 1024, 88 % vs 68 % at

strategy	budget 512	budget 1024	budget 2048
no_eviction	100 %	100 %	100 %
recency	0 %	0 %	0 %
streaming_llm	12 %	20 %	24 %
evoke_discard	12 %	20 %	24 %
evoke_breadcrumb	12 %	20 %	24 %
h2o	20 %	20 %	40 %
snapkv	44 %	68 %	84 %
infllm	**84 %**	76 %	68 %
evoke_kv_restore	76 %	**88 %**	**88 %**
evoke_attention	76 %	**88 %**	**88 %**
evoke_recovery_aware	76 %	**88 %**	**88 %**

2048). Pure-recovery-less baselines (recency 0 %, the rest 12–24 %) confirm the "recovery is the differentiator" story is not Qwen-specific.

strategy	budget 512	budget 1024	budget 2048
no_eviction	100.0 % [86.7, 100.0]	100.0 % [86.7, 100.0]	100.0 % [86.7, 100.0]
infllm	**72.0 % [52.4, 85.7]**	52.0 % [33.5, 70.0]	52.0 % [33.5, 70.0]
evoke_kv_restore	52.0 % [33.5, 70.0]	56.0 % [37.1, 73.3]	**68.0 % [48.4, 82.8]**
evoke_attention	56.0 % [37.1, 73.3]	56.0 % [37.1, 73.3]	**68.0 % [48.4, 82.8]**
evoke_recovery_aware	56.0 % [37.1, 73.3]	56.0 % [37.1, 73.3]	**68.0 % [48.4, 82.8]**

Multifact on Llama 3.1 8B (n=5, Wilson 95% CI): The same crossover at the same budgets. `evoke_recovery_aware` tracks `evoke_kv_restore` within 4 pp at every budget on Llama, matching the within-CI behavior on Qwen. The single-needle and multi-fact stories therefore generalize across the two architectures with no scorer-layer tuning.

Agent_bench on Llama 3.1 8B. EVOKE strategies (`breadcrumb`, `kv_restore`, `attention`) and InfLLM all PASS the planted-config probe at every budget; recency, streaming_llm, evoke_discard, h2o all fail. Recovery latency on Llama 3.1 8B is systematically profiled in Section 7.1 (Llama row: `kv_block_load` 1.78–15.66 ms across block sizes 20–1280 tokens; re-prefill 12.58–236.89 ms over the same range).

7.14 Concurrency and PCIe bandwidth

A reviewer asked what happens to save/load bandwidth when multiple sessions evict at once. We drive $N \in \{1, 4, 8, 16\}$ concurrent sessions through the same evoke_serve instance (Qwen 2.5 7B, budget 1024, 14-turn planted-fact workload per session, ThreadPoolExecutor dispatch in `scripts/concurrency_bench.py`). Each session restores its own session ID via the `X-EVOKE-Session` header so KV state stays isolated; recovery primitives fire whenever the per-session budget trips.

N	probe_ok	total wall (s)	per-session wall p50 (s)	turn p50 (s)	turn p99 (s)
1	1/1	8.64	8.63	0.56	0.82
4	4/4	48.73	47.27	3.43	4.55
8	8/8	97.54	94.18	6.84	9.26
16	16/16	203.16	196.60	14.91	17.89

100% probe-OK at every concurrency level; the K/V splice primitive holds up to 16 concurrent evicting sessions without correctness loss. Per-session wall-clock scales roughly linearly with N (8.6 s at N=1 to 196.6 s at N=16, $\sim$23$\times$ at 16$\times$ concurrency), and per-turn p99 latency scales similarly (0.82 s to 17.89 s). The dominant scaling factor is GPU compute contention — a single 16 GB GPU runs one Qwen 2.5 7B forward pass at a time, so concurrent decodes serialize through model weights — not the PCIe save/load path. Back-of-envelope: at 56 KiB per token times 128-token blocks ($\sim$7 MiB per block), N simultaneous eviction events of k blocks each move $7Nk$ MiB across

PCIe Gen4 16 GB/s, ∼0.4 ms per block per session; at $N{=}16$ and a realistic 4-block eviction burst the aggregate PCIe traffic is 448 MiB at ∼28 ms serial worst case (parallelizable across CUDA streams). The PCIe pessimism in the reviewer's worry is not the binding constraint here; per-GPU compute throughput is. Production multi-tenant scaling — which is the next-paper concern (Section 9) — would need tensor-parallel or paged-virtual KV layout to lift the compute ceiling, which this paper does not claim to provide.

8 Discussion and Limitations

Hybrid memory plus thinking-mode interaction. On hybrid memory models, Qwen 3.5's `<think>...</think>` trace is by default stripped from the API response but kept in the physical KV cache. Strict alignment would require evicting the thinking range from the attention cache, but `llama_memory_hybrid::seq_rm` rejects partial tail rollback of the recurrent half (a Mamba state cannot be partially sliced) and aborts. EVOKE's eviction respects the rejection and the session resets on the divergent turn (one such reset is observed in the Qwen 3.5 demo at filler 9). The workaround that ships today is `EVOKE_SUPPRESS_THINKING_STRIP=1`: the server keeps the thinking trace in returned content, the client echoes it back, and the cached state stays aligned. A future no-shift eviction mode (using the attention-only `llama_kv_block_seq_rm/seq_add` primitives already in the fork) would let the thinking range be evicted from attention while leaving the recurrent state untouched.

Recurrent and pure-SSM models. EVOKE operates on the attention component of hybrid models. Pure-SSM models such as Mamba and RWKV are out of scope, since they have no traditional KV cache. The blurry memory of an SSM is preserved naturally on a hybrid model, because EVOKE does not touch it.

Memory accounting. Saved blocks live in host RAM. For Qwen 2.5 7B, one cell costs 56 KiB (28 layers $\times$ 2 (K and V) $\times$ 512 `n_embd_kv_gqa` $\times$ 2 bytes), so a 128-cell block is 7 MiB and a 1000-block session at this block size costs roughly 7 GiB of host RAM. `kv_restore_ram_budget_bytes` bounds this with an LRU on saved blocks; `kv_restore_spill_path` adds a disk-spill tier that catches LRU demotions before bytes are lost (Section 7.9). Both tiers are off by default.

Multi-session and the model-in-VRAM bottleneck. The session pool in Section 5 swaps KV state, but the model weights remain a single instance in VRAM. With a 7B Q4_K_M model consuming roughly 5 GB of VRAM, a 16 GB GPU has roughly 11 GB left for active KV, swap-in temporary state, and Flash Attention working memory. Concurrency is therefore bounded by per-session KV footprint, not by EVOKE's policy layer. Production multi-tenant deployments would need either tensor-parallel weight sharing or thinner per-session quantization to lift this ceiling.

Smart-recovery is a small retrieval system, by design. Policy-layer quality is bounded by the retrieval encoder's discrimination on the workload: a poor encoder picks the wrong blocks to recover, even though the bytes spliced back into the cache are still the model's own K and V. The architectural line between identity-keyed substitution and similarity-shaped selection is drawn once in Section 4.

The contribution boundary is the recovery primitive, not the eviction scorer. The $n{=}15$ multi-fact sweep (Section 7.6) and its Llama 3.1 8B replication (Section 7.13) show that the EVOKE-vs-InfLLM ranking flips along the budget axis. At tight budgets the win goes to whichever policy makes fewer selection failures, and a larger-K pure-retrieval recovery (the InfLLM adaptation at $K{=}8$) catches more blocks per turn than EVOKE's $K{=}4$ attention-driven selection; the per-fact failure cross-tab in Section 7.6 confirms the gap is selection-failure-dominated, not substitution noise. At loose budgets headroom lets a smarter scorer pay off and the attention-driven EVOKE configuration takes the top spot. The honest framing of the contribution is therefore: the recompute-free identity-keyed K/V splice is what divides recovery-bearing policies from recovery-less ones (the 20–40 % to 76–100 % gap on NIAH, the 0 % to 50–80 % gap on multifact), and the attention scorer is a regime-targeted improvement on top of the primitive rather than the load-bearing claim. The Llama capture-layer ablation in Section 7.13 reinforces this: changing the scorer's capture layer does not change the residency set on Llama at budget 1024, because selection is what the workload bottlenecks on.

Cross-architecture latency depth. Sections 7.1, 7.3, 7.4, 7.7, 7.10 and 7.11 measure latency and scorer ablations on Qwen 2.5 7B; Section 7.13 adds an end-to-end Llama 3.1 8B run covering NIAH, multifact, and agent_bench, confirming the regime crossover and recovery-latency story port without modification. Absolute latency on still-larger architectures (Mistral, DeepSeek, $\geq$ 70B parameter models) will shift with model size, GQA grouping, and quantization; the mechanism is architecture-agnostic at the primitive layer (Section 3).

Embedding quality. Smart-recovery scoring uses `bge-small-en-v1.5` (384-dim), sufficient for the NIAH workload in Section 7.5. Larger retrieval encoders (e.g., `bge-large`, `e5-mistral`) would presumably widen the discrimination band further on paraphrase-heavy probes or non-English content. The specific embedder is swappable via `EvokeConfig.use_retrieval_embeddings`.

Smart-recovery placement. Recovery currently appends blocks at the cache tail before the new user message is decoded, so recovered blocks land as earlier context the model attends back to from the freshest probe. This fits the chat-completions pattern. For streaming agent workloads where the model is generating tokens that subsequently need earlier context, an in-place insertion path (using the attention-only `llama_kv_block_seq_rm`/`seq_add` primitives) would let recovered blocks slot back at their original logical position.

9 Future Work

- **Paged-substrate port.** All numbers in this paper come from a flat unified KV cache (llama.cpp). Production LLM serving systems (vLLM with PagedAttention (Kwon et al., 2023), SGLang with RadixAttention (Zheng et al., 2024), LMCache (LMCache Team, 2024)) allocate KV across non-contiguous physical pages indexed by a virtual page table, which enables prefix sharing across concurrent requests but means the `kv_block_save`/`kv_block_load` primitives must reconstruct a block's K and V from page-resolved scatter/gather rather than a contiguous read. The algorithmic content (typed memcpy of K and V keyed by layer/head/range, then RoPE re-anchoring) is unchanged; the engineering work is the page-table-resolved I/O, multi-tenant scheduling against the host's eviction policy, and PCIe-bandwidth scheduling at high concurrency. We name this as the natural successor work.
- **iSWA end-to-end on Gemma.** The fork-side `llama_kv_block_save` and `llama_kv_block_load` now handle both base and SWA sub-caches via `llama_get_kv_cache_swa()` (fork commit `1f37e75e7`). The remaining engine-side bookkeeping for the dual-cache surface and a Gemma 3 or 4 GGUF end-to-end smoke through the existing demo and agentic-eval scripts is the next step.
- **No-shift eviction mode for hybrid plus thinking.** Use the attention-only `llama_kv_block_seq_rm` and `llama_kv_block_seq_add` primitives (already in the fork) to evict attention cells without compacting, so a hybrid model's recurrent state stays in sync while the thinking range is dropped from attention.
- **Multi-layer scorer.** The fork already exposes `llama_attn_capture_set_layers` for up to 16 layers per decode. The current scorer uses a single mid-layer; a more principled scorer could average across an early, a middle, and a late layer to pick up both syntactic and semantic relevance signals.
- **Tensor-parallel and quantized variants.** Validate that the C primitives work unchanged across multi-GPU tensor-parallel layouts and across the full Q3 to Q8 quantization range.
- **Continuous learning of harness priorities.** Today `evoke_priority` is set by the harness. A reasonable next step is for EVOKE to learn priority from observed attention patterns, so a harness that does not set priorities still gets the benefit.
- **Multi-fact, judge-scored, randomized-position bench.** Section 7.6 already exercises five facts per session with a model judge; lift this to a RULER- or LongBench-style suite with per-position jitter and partial-credit scoring so the metric is not a binary single-probe hit.

10 Conclusion

EVOKE treats the KV cache as a small fast tier of a memory hierarchy. Blocks the model is not currently attending to leave the cache to host RAM at metadata cost, and when a future turn needs them they are spliced back as their original K and V tensors with one RoPE phase shift. The recovery primitive runs in 0.5 to 7 milliseconds across block sizes from 20 to 1280 tokens, 20 to 32 times faster than re-prefilling the same tokens through the model (5.9 to 7.5$\times$ over the full save+load lifecycle, factoring in the eviction-side save cost). On a 14-turn planted-fact session (n=15), it matches the unconstrained `no_eviction` baseline's recall at 4$\times$ smaller cache footprint and at `truncate`-comparable wall-clock (21.20s [21.07, 21.33] vs 22.11s [19.46, 24.76]; `truncate`'s SSH-launch jitter dominates its CI so the two are not statistically distinguishable at α=0.05). Across the head-to-head comparisons, the recovery-bearing policies (`evoke_kv_restore`, `evoke_attention`, and the same-substrate `infllm` adaptation) recover the lost fact while every recovery-less baseline (recency, StreamingLLM, H2O, SnapKV) cannot — the distinction at depth is presence of a recovery primitive, not the smartness of the eviction scoring. The eviction-scoring choice is regime-dependent: at tight budgets, where selection failures dominate, a larger-K pure-retrieval recovery (the InfLLM adaptation) is competitive with or beats the attention-driven scorer; at loose budgets, where headroom lets a smarter scorer pay off, the attention-driven EVOKE configuration takes the top spot (Sections 7.6 and 7.13). The load-bearing contribution is therefore the recompute-free recovery primitive itself, with the attention scorer as a regime-targeted improvement on top of it.

Code and Reproducibility

The EVOKE policy layer (Python) is at `github.com/Anyesh/EVOKE` under Apache 2.0. The forked llama.cpp with the C primitives is at `github.com/Anyesh/llama.cpp` under MIT (the upstream license). Every number in Section 7 was produced by a script in `scripts/` checked into the EVOKE repository, with the exact invocations listed in Appendix A. Raw output files for the agentic eval and the keepalive eval are checked in under `results/`.

References

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS 2023*.

Rae, J. W., Potapenko, A., Jayakumar, S. M., & Lillicrap, T. P. (2019). Compressive Transformers for Long-Range Sequence Modelling. *arXiv preprint arXiv:1911.05507*.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing 568:127063, 2024*.

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*.

Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Han, S., & Sun, M. (2024). InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv preprint arXiv:2402.04617*.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., & Chen, B. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS 2023*.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient Execution of Structured Language Model Programs. *NeurIPS 2024*.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., & Chen, D. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS 2024*.

Feng, Y., Lv, J., Cao, Y., Xie, X., & Zhou, S. K. (2024). Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *arXiv preprint arXiv:2407.11550*.

Cheng, J., Liu, Y., Wang, K., Xiang, X., Yu, X., Liu, J., Yi, F., & Jiang, J. (2024). LMCache: 7x Throughput Improvement Through Layered, Cross-Engine KV Cache. *vLLM Production Stack project*.

Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., Li, S., Ji, Z., Xie, T., Li, Y., & Lin, W. (2024). Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. *arXiv preprint arXiv:2401.02669*.

Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). C-Pack: Packed Resources For General Chinese Embeddings. *SIGIR 2024*.

Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? *COLM 2024*.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., & Li, J. (2023). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *ACL 2024*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL 2024*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022*.

Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2-bit Quantization for KV Cache. *ICML 2024*.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., & Xiao, W. (2024). PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.

Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., & Han, S. (2024). Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *ICML 2024*.

Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., & Han, S. (2024). DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *arXiv preprint arXiv:2410.10819*.

Prabhu, R., Nayak, A., Mohan, J., Ramjee, R., & Panwar, A. (2024). vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *arXiv preprint arXiv:2405.04437*.

Qwen Team. (2024). Qwen 2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

A Reproducing the Numbers

All numbers in Section 7 were produced on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1, Python 3.12) against Qwen 2.5 7B Instruct (`Qwen2.5-7B-Instruct-Q4_K_M.gguf` from the official Qwen GGUF release). The EVOKE policy layer runs in Python; the C primitives live in the forked llama.cpp.

A.1 Build the fork

```
# Clone the EVOKE-modified llama.cpp fork
git clone https://github.com/Anyesh/llama.cpp.git llama.cpp
cd llama.cpp

# Build with CUDA + shared library output
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
# Output: build/bin/libllama.so
```

A.2 Install the EVOKE Python package

```
git clone https://github.com/Anyesh/EVOKE.git evoke
cd evoke
uv sync --extra server

# Point the engine binding at the EVOKE fork build
export LLAMA_CPP_LIB=/abs/path/to/llama.cpp/build/bin/libllama.so
export EVOKE_MODEL_PATH=/abs/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf
```

A.3 Section 7.1 latency table (`profile_recover.py`)

```
uv run python scripts/profile_recover.py
```

The script does five warmup cycles at 40 tokens to settle CUDA graph compilation, then sweeps block sizes 20, 40, 80, 160, 320, 640, 1280 with five trials each, reports the mean save time, mean load time, and mean re-prefill time. The numbers in Section 7.1 are the means.

A.4 Section 7.2 eviction demo (`eviction_demo.py`)

```
# In one terminal: start the server
LLAMA_CPP_LIB=$LLAMA_CPP_LIB EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_HOST=0.0.0.0 EVOKE_BUDGET=1024 EVOKE_MODEL_NAME=qwen25 \
uv run python scripts/evoke_serve.py

# In another terminal: drive the demo
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/eviction_demo.py
```

For the Qwen 3.5 hybrid run, add `EVOKE_SUPPRESS_THINKING_STRIP=1` to the server env and load `Qwen3.5-9B-Q4_K_M.gguf`.

A.5 Section 7.3 head-to-head baselines (`baseline_bench.py`)

```
EVOKE_SERVER='http://eval-host:8000' \
EVOKE_SSH_HOST='user@eval-host' \
EVOKE_LIB_PATH='/path/to/libllama.so/on/eval/host' \
EVOKE_MODEL_PATH='/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf' \
EVOKE_BUDGET=1024 EVOKE_N_CTX=16384 EVOKE_MODEL_NAME=qwen25 \
  uv run python scripts/baseline_bench.py
```

The bench drives three policies (`no_eviction`, `truncate`, `evoke`) over the same 14-turn conversation via SSH-launched server instances. The inter-policy launch wrapper is in `scripts/` (commit `2b3cbea`).

A.6 Sections 7.4 and 7.7 agentic eval (`agent_bench.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench.py
```

The script runs five strategies (`recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`) at each budget. For the Section 7.7 attention row, set `EVOKE_W_ATTENTION=0.5` and `EVOKE_ATTN_LAYER=20`, which enables the `evoke_attention` strategy.

A.7 Section 7.10 keepalive workload (`agent_bench_keepalive.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench_keepalive.py
```

The keepalive variant has each filler-file content reference the central `config.py` so the model's attention keeps coming back to the config block, which is what the attention scorer exploits.

A.8 Multi-session pool sanity check (`verify_multi_session.py`)

```
# Server must already be running (see A.4)
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/verify_multi_session.py
```

The script drives two sessions with distinct planted codewords through interleaved chat requests and asserts that each session retrieves its own codeword, confirming state-swap correctness.

B The Fork

The llama.cpp fork is hosted at `github.com/Anyesh/llama.cpp`. The fork tracks upstream `ggml-org/llama.cpp` and adds the EVOKE-specific commits listed below.

B.1 EVOKE-specific commits in the fork

Commit	Title	What it adds
`1f37e75e7`	EVOKE: iSWA dual-cache support in `kv_block_save/load` (#41)	`llama_get_kv_cache_swa()` and an extended save/load buffer layout that covers both base and SWA sub-caches on Gemma 3 and 4
`151f1cf93`	EVOKE: multi-layer attention capture (up to 16 layers per decode)	`llama_attn_capture_set_layers(ctx, layers, n)` writes one slab per layer in a single decode
`713f202e8`	EVOKE: attention-weight capture primitive	The original single-layer side-compute path: `llama_attn_capture_set_layer`, `_set_buffer`, `_get_dims`, `_get_written`
`9b6232446`	EVOKE: attention-only `seq_rm` and `seq_add` primitives	`llama_kv_block_seq_rm` and `llama_kv_block_seq_add` reach the attention sub-cache directly via `llama_get_kv_cache`, for "no-shift" eviction modes
`a29599f4c`	EVOKE: `kv_block` primitives now cover hybrid memory and mrope models	Extends `llama_get_kv_cache` to unwrap `llama_memory_hybrid` via `get_mem_attn()` and `llama_memory_hybrid_iswa` via `get_mem_attn()->get_base()`. Removes the mrope `seq_add()` and `get_can_shift()` guards.

B.2 The two recovery primitives, verbatim

From `include/llama.h`:

```
LLAMA_API size_t llama_kv_block_save(
    struct llama_context * ctx,
            llama_seq_id   seq_id,
               llama_pos   p0,
               llama_pos   p1,
                 uint8_t * dst,
                  size_t   cap);

// Load a buffer produced by llama_kv_block_save into seq_id starting at new_p0.
LLAMA_API bool llama_kv_block_load(
    struct llama_context * ctx,
             llama_seq_id   seq_id,
           const uint8_t * src,
                  size_t   size,
               llama_pos   new_p0);
```

`save` returns the number of bytes written (or zero on error, e.g. `cap` too small). `load` returns true on success. The caller is responsible for the buffer allocation in both directions.

B.3 Build instructions

The fork builds cleanly against the upstream llama.cpp toolchain. Requirements: CMake $\geq$ 3.14, a C++17 compiler, and (for the GPU path) CUDA Toolkit 11.8 or later (we tested with 13.1).

```
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
```

The Python binding (`evoke/_engine_lib.py` and `evoke/llama_engine.py`) loads the shared library indicated by the `LLAMA_CPP_LIB` environment variable (a file path). It then derives `LLAMA_CPP_LIB_PATH` (a directory) for `llama-cpp-python` 0.3.x compatibility so a single loaded module backs both the Python wrapper and the EVOKE ctypes binding. Mixing two builds (the upstream wheel and the fork) causes layout-mismatch crashes; using a single fork build everywhere is the supported configuration.

B.4 Attention-capture C API surface, verbatim

The full C API for attention-weight capture, summarized in Section 3.3, added to `include/llama.h`:

```
LLAMA_API void   llama_attn_capture_set_layer (llama_context *, int32_t layer);
LLAMA_API void   llama_attn_capture_set_layers(llama_context *, const int32_t *
    layers, int32_t n);
LLAMA_API void   llama_attn_capture_set_buffer(llama_context *, float * dst,
    size_t cap_floats);
LLAMA_API void   llama_attn_capture_get_dims  (const llama_context *, int32_t *
    n_layers,
                                               int32_t * n_q, int32_t * n_h,
    int32_t * n_kv);
LLAMA_API size_t llama_attn_capture_get_written(const llama_context *);
```